# Assignment 1 Report

Aadil Goyal

October 24, 2026

**Abstract**

The GitHub Repo can be found at link and the STM Board used is stm32f103c8t6

## 1 Question 1

Question: Cats or Dogs?

Ans: Not really into either, but if I had to choose then Dogs

## 2 Question 2

Question: What is the difference between UART and USART? Would you prefer working with UART or USART while working on the rover (imagine you want to communicate between the Raspberry Pi and an STM32 bluepill to transfer motor commands and telemetry data)? Explain why.

Ans: UART stands for Universal Asynchronous Receiver Transmitter while USART stands for Universal Synchronous Asynchronous Receiver Transmitter
More differences are as follows.

| Feature | UART | USART |
|---|---|---|
| Full Form | Universal Asynchronous Receiver Transmitter | Universal Synchronous/Asynchronous Receiver Transmitter |
| Communication Type | Asynchronous only | Supports both Synchronous and Asynchronous |
| Clock Signal | Not used | Uses an additional clock line in synchronous mode |
| Number of Pins Used | 2 (TX and RX) | 3 in synchronous mode (TX, RX, and CLK) |
| Data Sampling | Based on start and stop bits (internal clock) | Data is sampled on the rising or falling edge of external clock |
| Application Example | Basic serial communication with PCs, Bluetooth modules | Smartcard communication, SPI-like data transfers |

I would prefer working with USART because it can give higher data transfer rates and better reliability. This is extremely important for transferring motor commands and telemetry data inside the rover.

# 3 Question 3

Question: If I set the TIMEOUT parameter of HAL UART Receive to 0, what would happen? Hint: Would the code still be blocking?

Ans: If we set TIMEOUT to 0, then it will do the check for a signal instantly and if there is no signal in the buffer, then it would give a timeout. This would make it non-blocking. However, it only has time to read as much data as the buffer can hold, which is only 1 byte in case of our board.

# 4 Question 4

Question: Explain all the code you have written briefly.

Ans:

We first have a bool *isLonely*, if it is *true* then we have not received, a signal for 5+ seconds and have already transmitted a lonely msg, so now we check only for 2sec Is *ifLonely* is *false*, we check for 5secs

Now we check if we have received a response or not, if we ran into a timeout into a timeout then we will set *isLonely* to *true* and transmit the lonely msg If we received a msg, then we first put *isLonely* to *false* and then check whether it is *'1'*, *'0'* or something else. Depending on this we turn inbuild led ON or OFF or tranmit an invalid data msg

This code uses polling as we are calling the *HAL_UART_Receive* function and waiting for a signnal until *TIMEOUT* and hence it is also blocking

# 5 Question 5

Question: Explain how interrupts work for transmission and how it is different from interrupts for receiving.

Ans: In a Tx Interrupt, we start sending the first byte, once it is sent, the CPU is interrupted and the next byte is then sent. So we only use the CPU while starting thhe transmission of a byte and for the remaining time, the CPU is free and not blocked.

In a Rx Interrupt, you set how many bytes you want to receive and then wait for them. When that number of bytes is received, then the Rx CallBack function is called. Hence It is also non blocking as while waiting for the bytes it does not block the CPU

# 6 Question 6

Question: Explain the logic you have used for implementing the task using interrupts briefly.

Ans: We first call *HAL_UART_Receive_IT*, so as soon as we get a signal, we go the the Rx Handler (*HAL_UART_RxCpltCallback*) and check whether the msg is *'0'* or *'1'* or something else. Depending on it we set the inbuilt led to OFF or ON or transmit "Invalid Data". Finally we update the *last_time* to the current time and *isLonely* to *false*

Parallely in the while loop, we first check *isLonely*. If *isLonely* is *false* (we just received a signal), so we check whether it has been 5 secs since that happened. If it did then we transmit the lonely msg and switch *isLonely* to *true* and *last_time* to rn. If *isLonely* is *true* (we are now in the 2 sec stage), so we check whether it has been 2 secs. If it has been then we transmit a lonely msg and update *last_time*.

# 7 Question 7

Question: Explain how DMA works for transmission and how it is different from DMA for receiving.

Ans:

The process is extremely similar to that in Interrupt except that the CPU involvement is even less than that in Interrupts.

During Tx DMA, the first byte is asked by the UART to the DMA and then the DMA sends that byte which is copied into the UART's data registers, then the next one and so on. After the transmission is complete, the Tx CallBAck is called and the CPU is involved.

During Rx DMA, as soon as the UART receives a byte, the DMA asks to move it into our buffer. During this process the CPU is not involved. Once the required number of bytes have been moved, the Rx CallBack is called and the CPU is interrupted.

# 8 Question 8

Question: Explain the logic you have used for implementing the task using DMA briefly.

Ans: For this, I have used the exact same code as for interrupt, only changed IT to DMA

# 9 Question 9

Question: How does *Serial.print* and *Serial.read* work for an Arduino microcontroller? Explain in the context of blocking and non-blocking techniques.

Ans:

*Serial.read* is a non-blocking function, it reads one byte from the buffer (the oldest out of the new ones), if there is no new data, then it gives -1. If a lot of data is received or we are reading data slowly, the buffer can get full and starts updating unread bytes with newly received bytes.

*Serial.print* moves the data into a buffer and transmits data one byte at a time. If the buffer gets full, then *Serial.print* waits. It does not discard any data and hence can be blocking.

# 10 Question 10

Question: Explain the following terms:
• Prescalar
• Auto-reload register (ARR)
• Capture/Compare register (CCRx)
and how they are involved in PWM generation. Give formulae for the duty cycle and PWM frequency based on the above terms.

Ans:
Prescalar denotes after how many clock cycles the timer counter increments by 1
Auto-reload register (ARR) is till where the timer counter goes before being reset to 0
Capture/Compare register (CCRx) denotes the value with which the counter is compared. If counter is below it then pin is set to high, if counter is above the pin is set to low.

$$\text{DutyRatio} = \frac{\text{CCRx}}{\text{ARR} + 1} \tag{1}$$

$$f_{\text{PWM}} = \frac{f_{\text{clk}}}{(\text{PSC} + 1)\,(\text{ARR} + 1)} \tag{2}$$

# 11 Question 11

Question: Explain the code you have written for implementing this task briefly.

Ans: For this, I have written only one line of code in the main function which starts the PWM generation.
Now the timer counter is incremented after PSC+1 clock cycles which inn this case is 63
So every 64 cycles, the counter i incremented
The counter is incremented until it reaches ARR+1, which in this case is 999
So it counts till 1000 and then rests back to 0
While the counter is below CCR (which is 300), the signal is high then it is low. So it is high for 300 out of 1000 and then low, so it gives about 30% duty cycle.
The reason for +1 is because counter starts from 0.